Chapter II : Packing Information into Name

Group 15:

LY Chhaythean
MIN Sivee
TIENG Sopanha

October 19th, 2026

Introduction

whether for a variable, function, or class, function should look like a “tiny comment". Because the overarching goal of readable code is to minimize a reader's "time-till-understanding," choosing a good name offers a high-return opportunity to convey maximum information in a very small footprint.
Rather than relying on vague or "empty" words (such as get, size, or tmp) that force readers to pause and decipher intent, the authors argue that naming should be an intentional act of packed communication.

Technique

1. Choose specific words instead of empty ones (FetchPage(), not GetPage()).
2. Avoid generic names like tmp and retval, or know when they’re acceptable.
3. Use concrete names instead of abstract ones (CanListenOnPort(), not ServerCanStart()).
4. Attach extra information to a name with a prefix or suffix (delay_ms, unsafe_url).
5. Decide how long a name should be based on its scope.
6. Use name formatting (capitals, underscores) to convey meaning.

Core philosophy

The "Tiny Comment" Principle

- ***The Goal:*** Minimize the time it takes for someone else to understand your code.
- **The Rule:** A name should act like a "tiny comment" that packs as much specific information as possible into a small space.
- **The Mindset:** If a reader has to look at the code below the name to figure out what the name means, the name has failed.

Technique & Concept

1.CHOOSE SPECIFIC WORDS

- *Instead of `GetPage()`: Use `FetchPage()` or `DownloadPage()` (Clarifies the source/method)*

- *Instead of `Size()`: Use `Height()`, `NumNodes()`, or `MemoryBytes()` (Clarifies the dimension)*

- *nstead of `Stop()`: Use `Kill()` (irreversible)or `Pause()` (resumable) (Clarifies the outcome)*

Technique & Concept

2.AVOID GENERIC NAME

- *Generic names are often cop-outs.*
- *`retval`: Replace with the actual value being accumulated (e.g., `sum_squares`, `user_count`).*
- *`tmp`: Only use if "being temporary" is the single most important detail.*
- *Good: A 3-line variable swap.*
- *Bad: A variable used across 20 lines just because you didn't want to think of a real name.*

```python
def calculate_student_average(student_scores: list[float]) -> float:
    if not student_scores:
        raise ValueError("Student scores cannot be empty.")

    total_score = sum(student_scores)
    student_count = len(student_scores)
    average_score = total_score / student_count

    return average_score


scores = [85.0, 90.0, 78.0, 92.0]
average_score = calculate_student_average(scores)

print(f"Student average: {average_score:.2f}")
```

3. Concrete over Abstract

- **Meaning:** Describe exactly what the thing does, not a general idea of it.
- **Functions**: Name the real action (e.g., CanListenOnPort() instead of ServerCanStart()).
- **Macros**: State what is blocked, not an opinion (e.g., DISALLOW_COPY_AND_ASSIGN instead of DISALLOW_EVIL_CONSTRUCTORS).
- **Flags**: Name the behavior, not where it is used (e.g., --extra_logging instead of --run_locally).

```python
def can_listen_on_port(port: int) -> bool:
    return 1 <= port <= 65535

MAX_LOGIN_ATTEMPTS = 5

parser.add_argument(
    "--extra-logging",
    action="store_true",
)
```

4. Attach Extra Information

- Add a critical detail to the name so the reader always sees it.
- **Units**: Append the unit to measurements (e.g., elapsed_ms, delay_secs, size_mb).
- **Security**: Mark unsafe or raw data (e.g., plaintext_password, unsafe_url, unescaped_comment).
- **Encoding**: Show the data's format or state (e.g., html_utf8, data_urlenc).

```
elapsed_ms = 250
file_size_mb = 12.5
plaintext_password = "secret123"
html_utf8 = "<h1>Hello</h1>"
```

Technique & Concept

5. Match Name Length to Variable Scope

The ideal length of a name depends on how far it is used.

- *Short Scope: Variables spanning a few lines can use short names (e.g., m in a 4-line block). Context is immediately visible.*
- *Large Scope: Globals and class members need long, highly descriptive names. Declaration and usage are far apart.*
- *Prune Unneeded Words: If context is clear, drop redundant words (e.g., ToString() instead of ConvertToString()).*

```python
# TOO SHORT — "self.x" tells you nothing about the state

class PaymentProcessor:
    def __init__(self):
        self.n = 0
        self.x = 3
        self.c = GatewayClient()


# RIGHT SIZE — readable from any method, or from another module
class PaymentProcessor:
    def __init__(self):
        self._retry_count = 0
        self._max_retry_attempts = 3
        self._gateway_client = GatewayClient()
```

Technique & Concept

6. Use Formatting to Convey Meaning

- *Apply consistent formatting rules as visual syntax highlighting.*
- *Class Members: Append an underscore (e.g., offset_) to distinguish from local variables.*
- *Constants: Format as kConstantName to separate from #define MACRO_NAME.*
- *Constructors/Frameworks: Capitalize JS constructors (DatePicker) or prefix jQuery objects ($all_images).*

```
class ImageGallery {
 private:
    // Class Members: Appended with trailing underscore
    int offset_;
    double scale_factor_;

 public:
    // Constants: Standardized as kConstantName
    static const int kMaxImageLimit = 100;

    // Constructors: Capitalized name
    ImageGallery(int offset) : offset_(offset), scale_factor_(1.0) {}

    void ProcessGallery() {
        // Frameworks / jQuery: Prefixed with $
        auto $all_images = $("#gallery img");

        // Local variables: standard lower_snake_case without underscore
        int total_count = $all_images.length();

        if (total_count > kMaxImageLimit) {
            total_count = kMaxImageLimit;
        }

        offset_ = total_count * 2;
    }
};
```

2026

Thank You!

Q&A